

Openship Desktop

Architecture, onboarding & auth flows, and shared-package reference

Generated 2026-07-02

Contents

1. [Openship Desktop — Overview](#)
2. [Desktop onboarding, cloud auth, and the IPC bridge](#)
3. [Shared packages the desktop app depends on](#)

Openship Desktop

`apps/desktop` is an Electron **shell**, not a separate frontend. It contributes almost no UI of its own — it opens a `BrowserWindow` and points it at the same Next.js dashboard (`apps/dashboard`) that browser and CLI users see, and exposes a small native bridge (config storage, native dialogs, the system browser, cloud-auth polling) via `contextBridge` /IPC.

```
apps/desktop
├─ forge.config.json      # Electron Forge packaging config
├─ package.json           # @repo/desktop
├─ scripts/dev.mjs        # dev loop: tsc --watch + electronmon
├─ src/
│   └─ main/
│       └─ index.ts       # the entire main process (~650 lines, one file)
│           └─ types/desktop.d.ts # DesktopBridge type surface (partial/legacy)
│               └─ preload/
│                   └─ index.ts # contextBridge exposure -> window.desktop
└─ tsconfig.json
```

Everything the app does lives in two files: `src/main/index.ts` and `src/preload/index.ts` . There is no `src/renderer` — the renderer is whatever the dashboard's `/onboarding` page or dashboard root serves over HTTP.

Dependencies

```
"dependencies": {
  "@repo/core": "workspace:*",
  "@repo/onboarding": "workspace:*"
}
```

- **@repo/core** — supplies the four hardcoded runtime URLs (`CLOUD_API_URL` , `CLOUD_DASHBOARD_URL` , `LOCAL_API_URL` , `LOCAL_DASHBOARD_URL`). See [shared-packages.md](#).
- **@repo/onboarding** — supplies onboarding types (`SystemSettings` , `TunnelConfig`) and `buildSetupPayload` , plus (via the preload script only) `isPrivateIp` / `validateServerAddress` / `validateSshPayload` / `buildSshSettings` . See [shared-packages.md](#).

No other workspace packages are pulled in. The desktop app does not talk to the database, does not import `@repo/adapters` , and does not run any deployment logic itself — all of that lives in `apps/api` , which the desktop app treats as an opaque HTTP service.

Process model

Main process (`src/main/index.ts`):

- Owns a single `BrowserWindow` (`mainWindow`), created in `createWindow()` .
- Owns a tiny JSON config store (`ConfigStore` , backed by `app.getPath("userData")/config.json`).
- Registers every `ipcMain.handle(...)` channel the renderer can call.
- Makes all outbound HTTP calls to the API (`net.fetch`) — the renderer never talks to the API directly except through the dashboard's own `fetch` calls (which go straight over HTTP, unrelated to Electron).

Preload (`src/preload/index.ts`):

- Runs with `contextIsolation: true` , `nodeIntegration: false` , `sandbox: false` (window creation options in `createWindow()`).
- Exposes a single global, `window.desktop` , via `contextBridge.exposeInMainWorld("desktop", {...})` .
- Also `require()` s `@repo/onboarding` directly (not via IPC) to expose pure validation helpers synchronously to the renderer — wrapped in `try/catch` since this is "non-critical" per the code's own comment.

Renderer: whatever page is currently loaded — either the dashboard's `/onboarding` wizard or the dashboard root. The dashboard code checks `window.desktop` at a handful of call sites to branch between "running inside Electron" and "running in a plain browser" (see [onboarding-and-auth.md](#)).

Window lifecycle

`createWindow()` (`src/main/index.ts:190-253`):

1. Restores saved bounds from `ConfigStore` (`windowBounds`), else defaults to 1400×900, min 900×600.
2. macOS gets `titleBarStyle: "hiddenInset"` with custom traffic-light position; other platforms get the default title bar.
3. Window is created hidden (`show: false`) and only shown on `ready-to-show` , to avoid a white flash.
4. Decides what to load based on `store.get("onboardingComplete")` :
 - `false` → `loadOnboarding()` → loads `${LOCAL_DASHBOARD_URL}/onboarding` .
 - `true` → `loadDashboard()` → loads the persisted `dashboardUrl` ; if that URL fails to load, it resets onboarding state and falls back to onboarding again.
5. External links (`target=_blank / window.open`) are intercepted via `setWindowOpenHandler` and routed to the OS default browser via `shell.openExternal` , never opened as a second Electron window.
6. `did-navigate` is watched for one specific transition: if the window lands on `LOCAL_DASHBOARD_URL` root while onboarding isn't yet marked complete, that's treated as "onboarding just finished" and the config store is updated accordingly. This is a **secondary/legacy** completion signal — the primary signal, in the currently-shipped onboarding UI, is the explicit navigation to `desktop-login / desktop-claim` described in [onboarding-and-auth.md](#).
7. Window bounds are persisted on `close` .

A dev-only global shortcut, `Cmd/Ctrl+Shift+F12` , wipes onboarding state and reloads the onboarding page — useful for repeatedly testing first-run flows without clearing `userData` by hand.

Local config store

`ConfigStore` (`src/main/index.ts:66-103`) is a hand-rolled JSON store, not `electron-store` :

```
interface AppConfig {
  apiUrl: string;
  dashboardUrl: string;
  onboardingComplete: boolean;
  windowBounds?: { x, y, width, height };
  system?: SystemSettings; // SSH creds – local backup copy only
  tunnel?: TunnelConfig;
}
```

File location: `<app.getPath("userData")>/config.json` . Every `set()` call synchronously rewrites the whole file (`writeFileSync`) — fine at this scale, no debouncing/locking needed.

`system` (SSH host/user/auth) is kept here purely as a **local backup**; the API's `instance_settings / servers` tables (via `POST /api/system/setup` or `/api/system/onboarding`) are the actual source of truth. `system:get-settings` prefers reading from the API and only falls back to this local copy if the API is unreachable.

Build & packaging

Packaging is [Electron Forge](#) (`forge.config.json`):

Maker	Platforms	Output
<code>maker-squirrel</code>	Windows	installer (<code>openship</code> app name)
<code>maker-zip</code>	macOS, Linux	zip
<code>maker-deb</code>	Linux	<code>.deb</code>
<code>maker-dmg</code>	macOS	<code>.dmg</code> (<code>ULFO</code> format)

Packager config: app name `Openship` , executable `openship` , icon at `resources/icon` , `asar: true` .

```
npm run build      # tsc: src/ -> out/
npm run package    # build + electron-forge package (unpacked app, no installer)
npm run make       # build + electron-forge make (produces installers/zips above)
```

`main` in `package.json` points at `out/main/index.js` — the compiled output, not `src/`.

Dev workflow

```
pnpm dev:desktop  # from repo root – turbo filter for @repo/desktop
# or, inside apps/desktop:
npm run dev
```

`npm run dev` runs `tsc` once, then `scripts/dev.mjs`, which:

1. Spawns `tsc --watch --preserveWatchOutput` (recompiles `main / preload` on save).
2. After a 2s settle delay, spawns `electronmon`. (restarts the Electron process whenever `out/` changes).
3. On `SIGINT`, kills both children cleanly.

Important: the desktop app does **not** start the API for you. It assumes `apps/api` (and, for the dashboard-served onboarding/dashboard pages, `apps/dashboard`) are already running and reachable at `LOCAL_API_URL / LOCAL_DASHBOARD_URL` (`http://localhost:4000 / http://localhost:3001` by default). In practice this means running `pnpm dev:api` and `pnpm dev:dashboard` (or `pnpm dev:local`, which runs both) alongside `pnpm dev:desktop`. See [onboarding-and-auth.md § process wiring](#) for why this matters more than the in-code comments suggest.

Where to look next

- [onboarding-and-auth.md](#) — first-run onboarding UX, the cloud PKCE auth flow, the full IPC channel reference, and the actual (vs. documented-in-comments) security model.
- [shared-packages.md](#) — full reference for `@repo/core` and `@repo/onboarding`, the two workspace packages the desktop app depends on.

Desktop onboarding, cloud auth, and the IPC bridge

This covers everything that happens between "user opens the Openship app for the first time" and "user is looking at the dashboard" — plus the full `window.desktop` IPC surface and the security model behind it (including where the code's own comments describe an intended design that doesn't match what actually runs).

First-run decision tree

`createWindow()` checks `store.get("onboardingComplete")`:

- **Not complete** → load `${LOCAL_DASHBOARD_URL}/onboarding` (the dashboard's own onboarding wizard, served over plain HTTP — same page a browser user hitting `localhost:3001/onboarding` would see).
- **Complete** → load the persisted `dashboardUrl` directly.

The onboarding wizard itself lives in `apps/dashboard/src/app/(onboarding)/onboarding/`, not in the Electron app. It's a client-side step machine (`OnboardingStep`: `choose` → `selfhost-choice` → `ssh` → `tunnel` → `preferences` → `loading`, or `choose` → `loading` for the cloud path), driven by `nextStep` / `prevStep` from `@repo/onboarding`. Step components live in `_components/` (`choose-step`, `selfhost-choice-step`, `ssh-step`, `tunnel-step`, `preferences-step`, `loading-step`).

The wizard behaves **almost identically** whether it's running inside Electron or in a plain browser tab — it only branches on `window.desktop` at one point (the cloud auth sub-step, below). This is a deliberate design: the same onboarding code serves desktop, browser-based self-host setup, and (per `@repo/onboarding`'s CLI usage) the CLI's `openship setup health-polling`.

Path A: Self-hosted onboarding

Driven by `runSelfHostedFlow` in `apps/dashboard/.../_lib/loading-flow.ts`:

1. User fills in SSH connection details (`ssh-step`) and tunnel/build-mode preferences.
2. On submit, the dashboard builds the payload client-side using `@repo/onboarding`'s `buildSshSettings()` (raw form → `SystemSettings`) and `buildSetupPayload()` (`SystemSettings` / `TunnelConfig` / `buildMode/authMode` → `SetupPayload`).
3. The dashboard **POSTs directly to** `POST /api/system/onboarding` — a public, unauthenticated, *first-run-only* endpoint (`apps/api/src/modules/system/system.routes.ts`), gated server-side by "does the `servers` table already have a row?" (403 if so). It reuses the same `setup()` controller logic as `/api/system/setup`.
4. On success, the dashboard does a plain browser navigation: `window.location.href = "${apiBase}/auth/desktop-login"`.

This path never touches the Electron IPC bridge. The `onboarding:complete` IPC handler in `src/main/index.ts` (which builds a setup payload via `buildSetupPayload` and POSTs it to `/api/system/setup` with the internal token) exists in the code but is **not currently called** by the shipped onboarding UI for self-hosted setup — the dashboard's own fetch call to the public `/onboarding` route supersedes it. Worth knowing if you're debugging why breakpoints in `onboarding:complete` never hit.

Path B: Cloud onboarding (and cloud reconnect)

This is the one path that genuinely branches on `window.desktop`, and it's the same PKCE-style flow used in three places: initial onboarding, the login page's "Continue with Cloud," and Settings → reconnect cloud.

Sequence (Electron present)

1. Renderer calls `desktop.onboarding.cloudAuth()` (or `desktop.cloud.connect()` from settings — same shape, fewer side effects).
2. **Main process** (`onboarding:cloud-auth` handler):
 - Waits for the local API via `waitForApi(LOCAL_API_URL)` (polls `GET /api/health`, up to 30×1s).
 - Pushes `authMode: "cloud"` to the API (`POST /api/system/setup`, internal-token-authenticated) — only in the onboarding variant, not the settings-reconnect variant.
 - Generates `nonce`, `state` (CSRF), and a PKCE pair (`codeVerifier` / `codeChallenge = sha256(codeVerifier)`), all with `node:crypto`.
 - Registers the triple with the API: `POST /api/auth/desktop-auth-start` (internal-token-authenticated), body `{ nonce, state, code_verifier }`. The API stashes this in a **single in-memory variable** (`pendingNonce`), 5-minute TTL — see "single in-flight auth"

caveat below.

- Opens the system browser via `shell.openExternal` to `${CLOUD_DASHBOARD_URL}/authorize?callback=...&app=Openship+Desktop&machine=<hostname>&state=...&code_challenge=...&flow=desktop-cloud`.
- Returns `{ ok: true, nonce }` immediately — it does not block on the browser flow.

3. **Renderer** starts polling `desktop.onboarding.cloudAuthPoll(nonce)` (or `cloudConnectPoll`) every ~2s.

4. User authenticates in the system browser against the cloud dashboard, which redirects to `${LOCAL_API_URL}/api/auth/cloud-callback?code=...&state=...`.

5. **API** (`cloudCallback` handler) sees both `code` and `state` → this is the desktop PKCE branch: consumes `pendingNonce` via `validateDesktopState(state)`, exchanges `code` + the stashed `code_verifier` with the actual cloud SaaS (POST `${cloudRuntimeTarget.api}/api/cloud/exchange-code`), mirrors the cloud user locally, stores the cloud session, mints a local session, and calls `resolveDesktopAuth(nonce, token, expiresAt)`. It responds with a small HTML "you can close this tab" status page — it does **not** redirect anywhere, since this is a system-browser tab, not the Electron window.

6. **Main process poll handler** (`onboarding:cloud-auth-poll`) sees `status: "resolved"` with a one-time `claimCode`, then:

- Navigates the Electron window itself to `GET /api/auth/desktop-claim?code=...`, which sets the session cookie via `Set-Cookie` and redirects to the dashboard.
- Listens for the resulting `did-navigate` to the dashboard origin, and *then* flips `onboardingComplete = true` in the local config store.
- The **settings-reconnect variant** (`cloud:connect-poll`) does *not* navigate the window at all on `resolved` — the cloud session was already persisted server-side by `cloudCallback`, so it just tells the renderer to re-check `cloudApi.status()`.

Sequence (plain browser, no `window.desktop`)

The dashboard doesn't fall back to a "you can't do this without desktop" message — it does the PKCE dance itself: builds the same nonce/state/PKCE pair client-side (`preparePkceFlow`), opens the cloud authorize URL in a new tab (`window.open`, not `shell.openExternal`), and points the callback at `${apiBase}/auth/cloud-callback` — same endpoint, same flow, just without Electron's poll loop wrapping it (the browser tab itself ends up on the dashboard after the callback completes).

Single in-flight auth caveat

`pendingNonce` / `pendingClaim` / `activeNonce` on the API side are held in **single, in-memory variables**, not a map — the desktop/dashboard cloud-auth flow supports exactly one in-flight authorization attempt per API process at a time. Starting a second cloud-auth flow before the first resolves or expires will stomp the first one. This matches a single-user desktop/self-hosted deployment model; it would not survive a multi-instance API deployment, but that's out of scope for a `DEPLOY_MODE=desktop` process.

Full IPC reference (`window.desktop`)

Exposed by `src/preload/index.ts`; each entry is `ipcRenderer.invoke(channel, ...)` unless noted.

Namespace	Method	Channel	What it does
—	<code>isDesktop</code>	—	<code>true</code> , static — the flag the dashboard checks for "am I in Electron."
<code>config</code>	<code>get(key)</code> / <code>set(key, value)</code> / <code>getAll()</code>	<code>config:get</code> / <code>config:set</code> / <code>config:getAll</code>	Raw read/write of the local <code>ConfigStore</code> JSON.
<code>app</code>	<code>version()</code>	<code>app:version</code>	<code>app.getVersion()</code> .
<code>app</code>	<code>platform</code>	—	<code>process.platform</code> , read synchronously at preload time (not IPC).
<code>app</code>	<code>cloudUrls()</code>	<code>app:cloud-urls</code>	<code>{ api: CLOUD_API_URL, dashboard: CLOUD_DASHBOARD_URL }</code> from <code>@repo/core</code> .
<code>app</code>	<code>localUrls()</code>	<code>app:local-urls</code>	<code>{ api: LOCAL_API_URL, dashboard: LOCAL_DASHBOARD_URL }</code> from <code>@repo/core</code> .
—	<code>navigate(url)</code>	<code>navigate</code>	<code>mainWindow.loadURL(url)</code> — generic escape hatch.
<code>onboarding</code>	<code>complete(apiUrl, dashboardUrl, sshPayload?, buildMode?)</code>	<code>onboarding:complete</code>	Persists config, pushes settings to <code>/api/system/setup</code> , navigates to <code>desktop-login</code> . Not currently exercised by the shipped self-hosted onboarding UI (see Path A above) — the wiring exists but the dashboard bypasses it.

Namespace	Method	Channel	What it does
onboarding	openExternal(url)	onboarding:open-external	shell.openExternal(url) .
onboarding	cloudAuth()	onboarding:cloud-auth	Starts the cloud PKCE flow, sets authMode: "cloud" + buildMode: "auto" . Used by onboarding's cloud path.
onboarding	cloudAuthPoll(nonce)	onboarding:cloud-auth-poll	Polls once; on resolved , navigates the window through desktop-claim and marks onboarding complete.
onboarding	browseFile()	onboarding:browse-file	Native "Select SSH Key" open-file dialog; returns absolute path or null .
system	browseFolder()	system:browse-folder	Native "Select Project Folder" open-directory dialog.
system	getSettings()	system:get-settings	Reads from API (GET /api/system/setup , internal-token-authenticated) if apiUrl is set and reachable; else falls back to the local system config blob.
system	updateSettings(settings)	system:update-settings	Merges into local config and best-effort pushes to POST /api/system/setup (non-blocking — local save always succeeds even if the API push fails).
cloud	connect()	cloud:connect	Same PKCE mechanics as onboarding:cloud-auth , but does not push authMode / buildMode and does not navigate the window. Used from Settings → cloud reconnect.
cloud	connectPoll(nonce)	cloud:connect-poll	On resolved , does nothing but report status — the session was already stored server-side; renderer just refreshes cloudApi.status() .
—	reset()	app:reset	Wipes onboardingComplete / apiUrl / dashboardUrl / system / tunnel and reloads the onboarding page. Also bound to the dev shortcut Cmd/Ctrl+Shift+F12 .
utils	isPrivateIp , validateServerAddress , validateSshPayload , buildSshSettings	(none — direct require , not IPC)	Pure functions from @repo/onboarding , loaded synchronously into the preload script and exposed as plain function references. Falls back to no-op stubs if require("@repo/onboarding") throws.

Note: `src/main/types/desktop.d.ts` declares a much smaller `DesktopBridge` interface than what's actually exposed (missing `app.localUrls` , the whole `cloud` namespace, `system.getSettings` / `updateSettings` , `utils` , and gives `onboarding.complete` / `onboarding.browseFile` narrower signatures). Treat the preload script (`src/preload/index.ts`) as the source of truth, not this type file — it's out of sync with the actual implementation. The dashboard has its own, separate `desktop.d.ts` (`apps/dashboard/src/desktop.d.ts`) for typing `window.desktop` on that side.

Security model — as documented vs. as implemented

The main process's own comment block (`src/main/index.ts:109-121`) describes this model:

- Generated fresh each app launch (never persisted to disk)
 - Passed to the API process via `INTERNAL_TOKEN` env var at spawn time
 - Only Electron (parent) and API (child) share it in memory
 - API only listens on `127.0.0.1` in desktop mode (network-level protection)
 - Other local apps can't read another process's env vars (OS-level isolation)

Verified against the actual code, **steps 2 and 4 don't currently happen**:

- No spawn relationship exists.** There is no `child_process.spawn` / `fork` anywhere in `apps/desktop` that launches `apps/api` . The two are started independently (e.g. via `pnpm dev:api` + `pnpm dev:desktop` , or as separate processes in a packaged desktop distribution). The

`internalToken` Electron generates (`randomBytes(32)`) is never written to any environment the API process reads.

- **The API doesn't bind to loopback specifically.** `apps/api/src/index.ts` calls `serve({ fetch: app.fetch, port })` via `@hono/node-server` with no `hostname` option — it listens on whatever the runtime's default is, on every configured deploy mode, not just desktop.
- **What actually gates X-Internal-Token -protected routes in desktop mode** is `apps/api/src/middleware/internal-auth.ts` + `loopback-peer.ts`: when `DEPLOY_MODE=desktop` and `INTERNAL_TOKEN` is unset on the API (the expected desktop configuration), the middleware falls back to checking that the **TCP peer address** of the incoming request is loopback (`127.0.0.0/8` or `::1`) — a kernel-level fact about the socket, not the `Host` header, and not dependent on any shared secret. Electron's requests satisfy this simply because `net.fetch("http://localhost:4000/...")` is a loopback connection, same as any other local process would produce.
- For every **non-desktop** deploy mode (cloud, docker, bare), `INTERNAL_TOKEN` is mandatory and boot-checked (`apps/api/src/config/env.ts`) — the timing-safe token comparison in `internal-auth.ts` is the real gate there.

Net effect: in desktop mode, the `X-Internal-Token` header Electron sends on every internal call is currently **decorative** — it's checked against nothing, because the server-side `INTERNAL_TOKEN` env var is unset and the loopback-peer check is what actually lets the request through. This is a reasonable security posture for a single-user local desktop app (anything that can reach `localhost:4000` is already running as the same OS user), but it means the token/spawn-based model in the comments is aspirational, not descriptive. Anyone hardening this later should either (a) wire the token through a real spawn relationship and enforce it server-side even in desktop mode, or (b) delete the token plumbing and document the loopback-peer check as the actual/only mechanism.

How the API actually gets reached (and a documented gap)

Nothing in `apps/desktop` starts `apps/api`. The desktop app's entire assumption is: "something, somehow, is already serving the API at `LOCAL_API_URL` and the dashboard at `LOCAL_DASHBOARD_URL`." In this monorepo that's satisfied by running the relevant turbo dev targets side by side (`pnpm dev:local` runs both API and dashboard; add `pnpm dev:desktop` in parallel). A packaged end-user build would need its own supervisor process to start the API — `apps/api/scripts/build-release.ts` builds exactly that kind of self-contained bundle (API + dashboard + a `start.ts` supervisor), but that pipeline is for the **self-host migration wizard** (moving an existing instance to a fresh VPS), not for desktop app packaging. There is currently no equivalent "start.ts" wired into `forge.config.json` for the Electron build.

Shared packages the desktop app depends on

The desktop app itself is thin; almost everything it relies on comes from two workspace packages. This is a full reference for both, since understanding the desktop app means understanding what these actually provide.

@repo/core (packages/core)

The monorepo's pure, dependency-light shared kernel — types, constants, and pure functions with no I/O. Consumed by every app (api , dashboard , desktop , cli , web) and by packages/adapters / packages/db . ESM-only, single entry point src/index.ts (source is the import target for workspace consumers — no build step required). Barrel file re-exports: types , stacks , constants , system , utils , errors , service-routing , runtime-config , workspaces , languages , mail-server .

runtime-config.ts — what desktop actually imports

```
export const DEFAULT_PORT = { web: 3000, dashboard: 3001, api: 4000, saasDashboard: 3002, saasApi: 4100 };
const localhost = (port) => `http://localhost:${port}`;

export const LOCAL_WEB_URL      = localhost(DEFAULT_PORT.web);           // http://localhost:3000
export const LOCAL_DASHBOARD_URL = localhost(DEFAULT_PORT.dashboard);     // http://localhost:3001
export const LOCAL_API_URL      = localhost(DEFAULT_PORT.api);           // http://localhost:4000
export const CLOUD_DASHBOARD_URL = "https://app.openship.io";
export const CLOUD_API_URL      = "https://api.openship.io";
```

These are **hardcoded literals**, not read from environment variables at the desktop layer — apps/desktop/src/main/index.ts imports them, aliases them to DEFAULT_*, then reassigns straight through with zero override logic. If you need to point a dev build of the desktop app at different ports, you'd have to edit this file (or add env-var indirection — none currently exists here).

The file also defines a richer runtime-target system that desktop doesn't use directly but that's worth knowing about:

- OPENSHIP_TARGET env var (local | cloud-saas | local-saas) selects a row from DASHBOARD_RUNTIME_TARGETS ; an invalid value throws at module load.
- OPENSHIP_CLOUD_TARGET optionally overrides which target row cloud resolves to.
- runtimeTarget , cloudRuntimeTarget , dashboardRuntimeOrigins (flat CORS-allowlist-friendly list of every dashboard+api origin) are all derived from this table and used heavily inside apps/api .

Everything else @repo/core exports

Module	Highlights
constants.ts	APP_NAME , DEPLOYMENT_STATUSES , PRODUCTION_MODES , ENVIRONMENTS , DOMAIN_STATUSES , SSL_STATUSES , BUILD_ENV_VARS (CI/telemetry-off vars injected into build containers), billing catalog (PLANS , PLAN_IDS , CREDIT_PACKS , CREDIT_CONVERSION , validatePlanPriceIds()).
types.ts	Cross-app types: DeploymentStatus , Environment , Framework , BuildStrategy , DeployTarget , RuntimeMode , AdapterType , SleepMode , DomainStatus , SslStatus , billing (PlanId , SubscriptionStatus , UsageMetric), auth (UserRole , TeamRole), ApiResponse<T> , PaginatedResponse<T> .
stacks.ts	The framework/language registry — single source of truth for ~35 supported stacks (Next.js, Nuxt, SvelteKit, Django, Rails, Laravel, Spring Boot, Phoenix, Docker/Compose, etc.) and 12 languages, each with build/runtime images, detection rules, package managers. Also getBuildImage() , getRuntimeImage() , getStackDefaults() , getProjectType() , isTypicallyStatic() , TRANSFER_EXCLUDES (paths excluded when shipping project files).
system.ts	SYSTEM — operational limits/defaults: max projects, max concurrent builds, build timeout (30 min), SSE session limits/TTL, CLOUD_DOMAIN: "opsh.io" , SSL renewal cadence, env var length limits, pagination defaults.
utils.ts	slugify() , generateId(prefix?) , formatBytes() , formatDuration() , sleep(ms) .
errors.ts	AppError hierarchy (NotFoundError , UnauthorizedError , ForbiddenError , ValidationError , ConflictError , DeployError), safeErrorMessage(err) (caps + sanitizes error messages before logging, to avoid leaking credentials from

Module	Highlights
	libraries like ssh2).
service-routing.ts	normalizeServiceLabel , defaultServiceHostnameLabel , resolveServiceHostnameLabel — shared subdomain/hostname computation for compose/monorepo service deployments.
workspaces/	Monorepo-workspace detectors (pnpm, npm, rush, cargo, go, uv, elixir-umbrella, maven, gradle, dotnet-solution) — findMatchingDetectors() , parseWorkspaceManifest() .
languages/	Per-language manifest/dependency/port detectors (JS, Python, Go, Rust, Ruby, PHP, Java, Elixir, Docker) — collectDependencies() , detectPort() .
mail-server/	Pure routing-plan builder for the self-hosted mail server: buildMailServerRoutes(input) produces HTTP routes (mail.<domain> , api.mail.<domain> , autodiscover.<domain>) and DNS records (A/MX/SPF/DKIM/DMARC/CNAMEs) as data — no I/O.

Desktop only touches the four URL constants; everything else in the table is there because the desktop app shares a package with apps/api / apps/dashboard , not because desktop uses it.

@repo/onboarding (packages/onboarding)

A framework-agnostic package with no Electron/Node-runtime-specific dependencies — pure logic reused by the Electron main/preload processes, the dashboard's own onboarding wizard, and (in one small spot) the CLI. No single app "owns" onboarding logic; this package is the shared engine.

```
packages/onboarding/src/
├─ types.ts          # all shared type/interface definitions
├─ validation.ts     # validation + private-IP detection
├─ ssh.ts            # SSH payload -> SystemSettings normalization
├─ flow.ts           # step metadata + nextStep/prevStep state machine
├─ api-client.ts     # setup payload builder + fetch helpers
└─ index.ts          # barrel: export * from all of the above
```

Core types (types.ts)

```
export interface SystemSettings {
  serverName?: string;
  sshHost?: string;
  sshPort?: number;
  sshUser?: string;
  sshAuthMethod?: string;
  sshPassword?: string;
  sshKeyPath?: string;
  sshKeyPassphrase?: string;
  sshJumpHost?: string;
  sshArgs?: string;
}

export interface TunnelConfig {
  provider: "edge" | "cloudflare" | "ngrok";
  token?: string;
}

export interface SshPayload {
  host: string;
  user: string;
  method: "password" | "key" | "agent";
  serverName?: string;
  password?: string;
  keyPath?: string;
  passphrase?: string;
}
```

```

port?: number;
jumpHost?: string;
sshArgs?: string;
}

```

Also: `BuildMode` ("auto" | "server" | "local"), `OnboardingPath` ("cloud" | "selfhost"), `HostingMode` ("remote" | "local"), `OnboardingStep` ("choose" | "selfhost-choice" | "ssh" | "tunnel" | "preferences" | "loading"), `OnboardingState` , `SetupPayload` , `OnboardingPlatform` (an interface for injecting `openExternal` / `browseFile` / `fetch` — lets the same wizard code run against Electron's bridge or a plain browser).

Validation (`validation.ts`)

- `isPrivateIp(ip)` — strips a trailing `:port` , then matches `10.x` , `172.16-31.x` , `192.168.x` , `127.x` , `"localhost"` , IPv6 unique-local (`fc.../fd...`), or `:::1` . Used by the wizard's step machine to decide whether a tunnel step is needed after SSH (a private/local server needs a tunnel to be reachable; a public IP doesn't).
- `validateServerAddress(ip)` — `null` if valid, else an error string. Empty → "Please enter your server IP address"; fails `/^[\\w.\\-:]+$/` → "That doesn't look like a valid IP address". Note: a shape check, not real IP-format validation.
- `validateSshPayload(payload)` — `null` if valid, else an error string, in order: missing `host` → address prompt; invalid `host` → `validateServerAddress` 's error; `method === "password"` with no `password` → "Please enter your server password"; `method === "key"` with no `keyPath` → "Please enter the path to your SSH key". `method === "agent"` requires nothing extra.

`buildSshSettings(payload: SshPayload): SystemSettings ( ssh.ts )`

Normalizes a raw form payload into `SystemSettings` : `sshPort` defaults to `22` , `sshUser` defaults to `"root"` if falsy, `serverName` / `sshHost` / `sshAuthMethod` copied through, and `sshPassword` / `sshKeyPath` / `sshKeyPassphrase` / `sshJumpHost` / `sshArgs` are only included in the output if truthy on the input (sparse — omitted rather than set to `undefined` /empty string).

`buildSetupPayload(settings): SetupPayload ( api-client.ts )`

Input: `{ system?: SystemSettings; tunnel?: TunnelConfig; buildMode?: string; authMode?: string }` .

Behavior:

- `defaultBuildMode = settings.buildMode || "auto"` , `authMode = settings.authMode || "none"` — always present.
- If `system` given: `serverName` defaults to `null` (explicit, not omitted), `sshPort` defaults to `22` , `sshUser` defaults to `"root"` ; everything else copied through or omitted-if-falsy, same sparse pattern as `buildSshSettings` .
- If `tunnel` given: `tunnelProvider` / `tunnelToken` set (token only if truthy).
- With neither, payload is just `{ defaultBuildMode, authMode }` .

This is the exact shape the API's `setup()` controller (`apps/api/src/modules/system/setup.controller.ts`) expects on both `POST /api/system/setup` and `POST /api/system/onboarding` .

Same file also exports `pushInstanceSettings(opts)` (POSTs the payload, 10s timeout, returns a boolean, never throws) and `waitForApi(opts, maxAttempts=30, intervalMs=1000)` (polls `GET /api/health` until `res.ok`). **Desktop implements its own inline copies of both of these in `src/main/index.ts` rather than importing them** — the package versions exist and are used by the dashboard/CLI side, but desktop only imports `buildSetupPayload` itself plus the types.

Step flow (`flow.ts`)

`STEPS: Record<OnboardingStep, StepDef>` (id/title/subtitle for each of the 6 steps) and `STEP_ORDER` . `nextStep(current, state)` / `prevStep(current, state)` implement the wizard's branching logic (e.g. skip the tunnel step for a public server, skip straight to `loading` for the cloud path) — both can return `null` (flow complete / no earlier screen). This is what `apps/dashboard/src/app/(onboarding)/onboarding/page.tsx` drives its `step` state with.

Where each export is actually used

Export	Desktop (main/preload)	Dashboard onboarding UI	CLI
<code>SystemSettings</code> , <code>TunnelConfig</code> (types)	✓	✓	—

Export	Desktop (main/preload)	Dashboard onboarding UI	CLI
<code>buildSetupPayload</code>	✓ (main)	— (dashboard uses it too, but posts to a different route — see below)	—
<code>pushInstanceSettings</code> , <code>waitForApi</code>	— (desktop reimplements inline)	—	<code>waitForApi</code> only, in <code>opensip setup</code>
<code>isPrivateIp</code> , <code>validateServerAddress</code> , <code>buildSshSettings</code>	✓ (preload only, via <code>require</code>)	✓	—
<code>validateSshPayload</code>	✓ (preload only)	✓	—
<code>OnboardingStep</code> / <code>OnboardingState</code> / <code>nextStep</code> / <code>prevStep</code>	—	✓ (this is the wizard's actual state machine)	—
<code>BuildMode</code> , <code>SshPayload</code> , <code>TunnelConfig</code> (as UI types)	—	✓	—

The dashboard's self-hosted onboarding path does call `buildSshSettings` + `buildSetupPayload` client-side, but then POSTs the result straight to the public `POST /api/system/onboarding` route via its own `api.post(...)` helper — not through Electron IPC, and not via this package's `pushInstanceSettings` . See [onboarding-and-auth.md](#) for the full flow and why that matters.